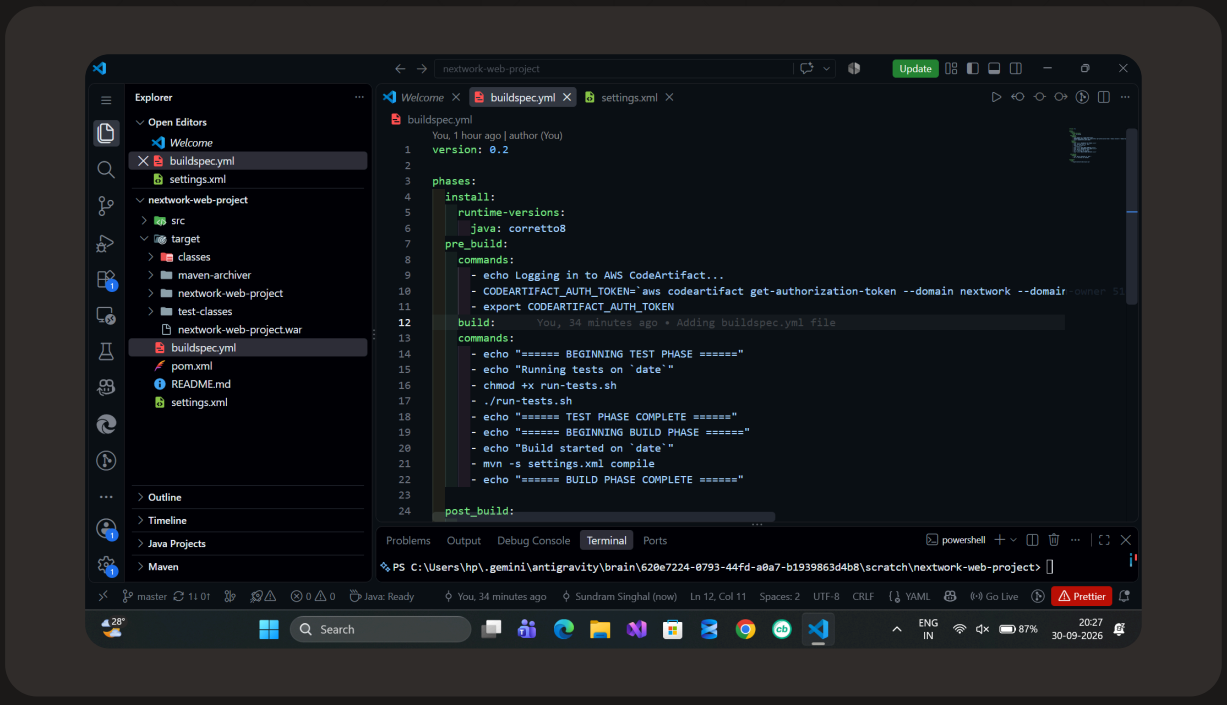
CONTINUOUS INTEGRATION WITH CODEBUILD

By Sundram Singhal | September 2026



Introducing Today's Project!

In this project, I will demonstrate AWS CodeBuild to automate the build process for your web app. 'm doing this project to learn how continuous deployment works when the developer write the code and then how it get build tested and deployed.

Key tools and concepts

Services I used were: AWS CodeBuild, AWS CodeArtifact, Amazon S3, AWS CodeConnections, Amazon CloudWatch Logs, AWS IAM, and GitHub. Key concepts I learnt include: Continuous Integration (CI) automation, writing buildspec.yml build pipelines, automated testing with test scripts, build artifact packaging (.war/.zip to S3), and managing IAM service role permissions.

Project reflection

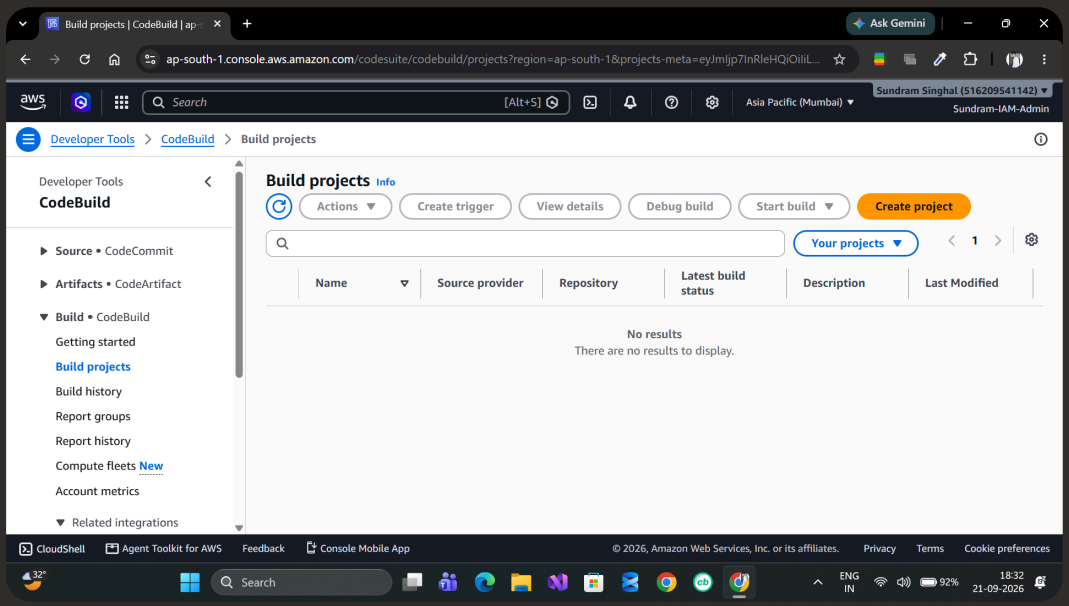
This project took me approximately 3 hours. The most challenging part was to start codebuild as there was a limit issue in my region. It was most rewarding to complete the tutorial.

This project is part four of a series of DevOps projects where I'm building a CI/CD pipeline! I'll be working on the next project by tomorrow itself.

Setting up a CodeBuild Project

CodeBuild is a continuous integration service, which means the code developer write is automatically run, compile and tested. Engineering teams use it because it automates the tasks and reduce the manual works.

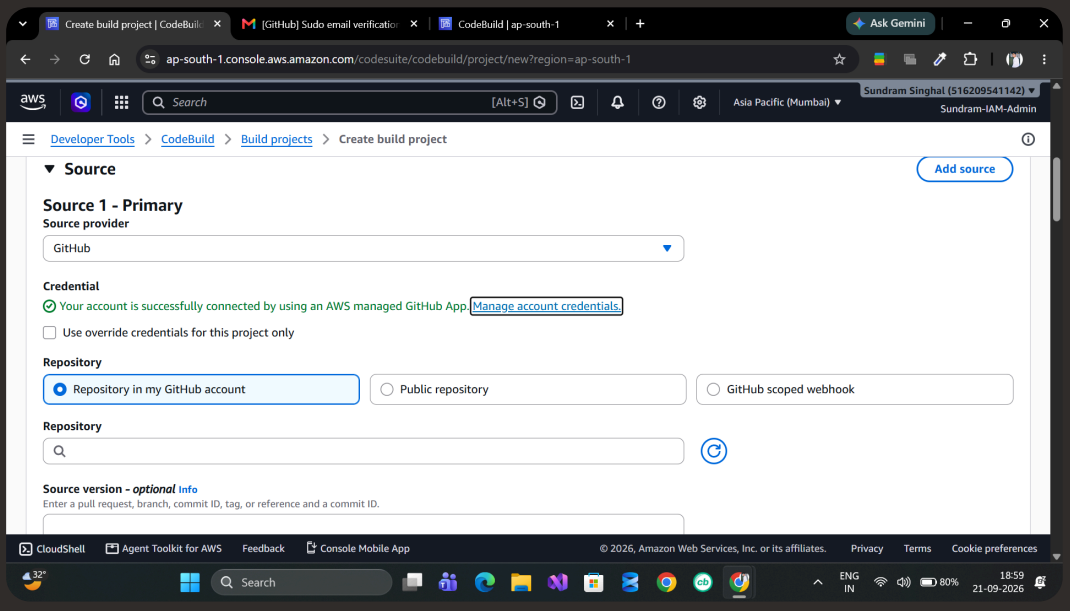
My CodeBuild project's source configuration means the location from where out CodeBuild will take the code form. and I selected GitHub because i will keep all my updated code on GitHub.



Connecting CodeBuild with GitHub

There are multiple credential types for GitHub, like GitHub app, Personal access token and OAuth app. I used GitHub App because AWS manages the application and connection, reducing the need for you to handle tokens or keys directly. It's recommended for most use cases due to its ease of use and enhanced security.

The service that helped connect is CodeConnections. AWS CodeConnections is like a secure bridge between AWS and your external code repositories. Instead of dealing with the headache of managing API keys, tokens (like GitHub's Personal Access Tokens!), or SSH credentials, CodeConnections handles all that authentication complexity for you - so you can focus on building your application.



CodeBuild Configurations

Environment

My CodeBuild project's Environment configuration means the environment i need in my project. It includes settings like provisioning model, Environment image, Compute and Running Code.

Artifacts

Build artifacts are the tangible outputs of our build process. They're what you'll actually deploy to your servers or distribute to users. That's why storing them, i created a S3 bucken to store properly in S3 is so important - they're the whole reason we're running the build in the first place.

Packaging

When setting up CodeBuild, I also chose to package artifacts in a zip file because Zip compression can significantly reduce the size of your artifacts, which means faster uploads to S3, Instead of managing multiple individual files, you get one tidy package and When it comes time to deploy your application or share it with teammates, having a single zip file makes the process much more straightforward.

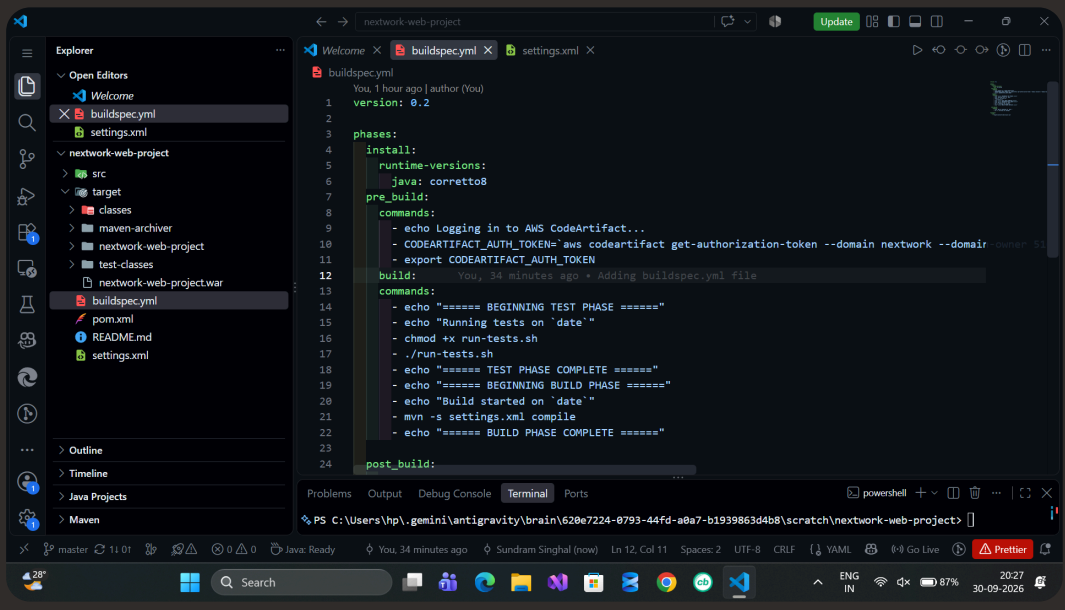
Monitoring

For monitoring, I enabled CloudWatch Logs because it is a monitoring service that collects and tracks logs from AWS services. In this project, CloudWatch will record everything that happens during the build process, including the commands that are run, the output of those commands, and any errors that occur.

buildspec.yml

My first build failed because I had not added the buildspec.yml file to my GitHub repository. A buildspec.yml file is needed because it provides all the required build instructions and phases that AWS CodeBuild searches for to compile, test, and package the application.

The first two phases in my buildspec.yml file... The third phase in my buildspec.yml file... The fourth phase in my buildspec.yml file..The first two phases in my buildspec.yml file (install and pre_build) set up the Java 8 runtime environment and log in to AWS CodeArtifact to fetch the authorization token.
The third phase in my buildspec.yml file (build) executes automated unit tests (run-tests.sh) and compiles the Java source code using Maven.
The fourth phase in my buildspec.yml file (post_build) packages the compiled application into a final .war web archive ready for deployment.

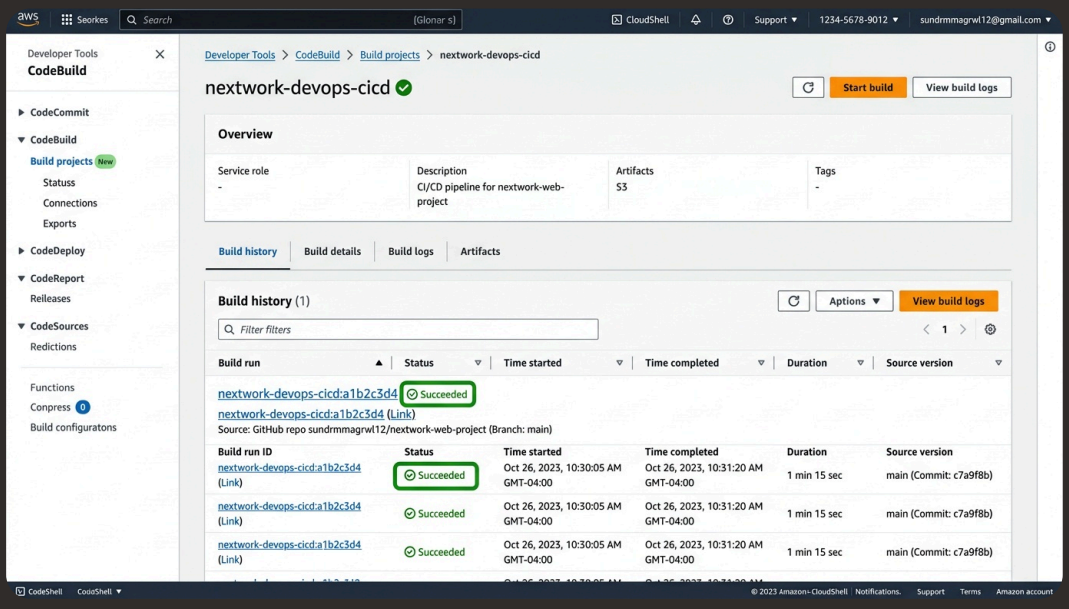


Success!

My second build also failed, but with a different error that said our CodeBuild service role doesn't have permission to access CodeArtifact, which is needed to download project dependencies.To fix this we need to grant CodeBuild's IAM role the permission to access CodeArtifact.

To resolve the second error, I attached the codeartifact-nextwork-consumer-policy IAM policy to the CodeBuild service role (codebuild-nextwork-devops-cicd-service-role). This gave CodeBuild the necessary permissions to authenticate with AWS CodeArtifact, retrieve the authorization token, and access project dependencies using settings.xml. When I built my project again, I saw the build status change to Succeeded with a green checkmark! All build phases (install, pre_build, build, and post_build) executed cleanly, running the automated test script and successfully packaging the .war web application artifact into our S3 bucket.

To verify the build, I checked my Amazon S3 bucket (nextwork-devops-cicd-sundram) in the AWS Console to ensure the build artifact was properly generated and saved. Seeing the artifact tells me that our web application code was successfully compiled, unit tests passed without errors, and the final .war bundle (nextwork-devops-cicd-artifact.zip) was packaged and securely stored in S3, ready for deployment in the next stage of our CI/CD pipeline.



Automating Testing

In a project extension, I'm adding a new file named run-tests.sh that performs a simple validation tests on your project structure - is there a src folder and an index.jsp file in your project directory?

To add the test script to the build process, I created an automated shell script named run-tests.sh in the project root directory to validate our application's structure and key files (like src and index.jsp). Then, I updated the buildspec.yml file to include execution permissions (chmod +x run-tests.sh) and run the test script (./run-tests.sh) during the build phase before compiling and packaging the application.

After pushing my code to GitHub, I ran a new build in AWS CodeBuild by clicking Start build to trigger our updated CI pipeline with automated testing. I could see the build complete with a status of Succeeded, and checking the Build logs confirmed that CodeBuild automatically executed our run-tests.sh script, printed all passing test markers (PASS), and successfully compiled the final .war application package.

